Full-Stack Graph Algorithm Engine & Traversal Simulator

Name: Parth Vaish **Enrollment no:** 24116066

1. Abstract

The Full-Stack Graph Algorithm Engine & Traversal Simulator is a comprehensive software suite engineered to analyze, benchmark, and visualize graph traversal algorithms in real-time. Moving beyond traditional console-based outputs, this project bridges a highly optimized C++17 mathematical backend with a responsive, HTML5 Canvas-based JavaScript frontend. By executing Dijkstra's Algorithm, A* Search, Breadth-First Search (BFS), and Depth-First Search (DFS) on unified datasets, the system provides microsecond-accurate benchmarking and step-by-step visual reconstructions. This dual-layered approach allows users to quantitatively analyze algorithmic time-complexity while qualitatively observing spatial exploration strategies.

2. Problem Statement & Objectives

Understanding graph algorithms purely through mathematical proofs or static code often obscures their real-world behavior. The objective of this project was to build a tool that answers the following questions:

1. **Execution:** How do different algorithms traverse the exact same spatial data?
2. **Efficiency:** What is the true execution time of these algorithms when compiled with extreme optimizations (-O2), and how do we measure times that are faster than standard system clocks?
3. **Visualization:** How can we visually differentiate the "thought process" of an optimal pathfinder versus a brute-force searcher?

3. System Architecture & The Data Bridge

The project strictly enforces the separation of concerns, decoupling heavy computations from UI rendering.

- **The Backend Engine (C++17):** Handles all memory management, random graph generation, coordinate mapping, and mathematical heuristic calculations. It operates purely in the terminal and uses the MinGW GCC compiler.

- **The Serialization Layer (JSON):** Because C++ and JavaScript cannot communicate directly in this environment, a custom JsonWriter class was built. It acts as an API, translating the C++ memory states (Nodes, Edges, Path Arrays, Execution Times) into a standardized results.json file.
- **The Frontend Visualizer (JS/HTML5):** A local Python web server (http.server) hosts a web dashboard. The JavaScript fetches the JSON payload, scales the coordinates dynamically to fit the user's screen, and orchestrates the animations.

4. Core Data Structures

To demonstrate a rigorous understanding of systems engineering, standard libraries were bypassed for critical components.

- **Adjacency List Graph (graph.h):** The graph is modeled as std::vector<std::vector<Edge>>. This structure is heavily optimized for sparse graphs. It ensures $O(1)$ constant-time access to any node's immediate neighbors, which is critical for the speed of BFS and DFS.
- **Custom Priority Queue (min_heap.h):** Dijkstra's algorithm relies heavily on finding the node with the lowest current cost. Instead of using the standard std::priority_queue, a custom Min-Heap was constructed from scratch. Utilizing a zero-indexed array, it enforces strict $O(\log N)$ extraction times via custom heapifyUp and heapifyDown tree-balancing functions.

5. Algorithmic Logic & Implementation

The system executes four distinct algorithms. Each is fed the exact same start (src) and end (dest) nodes to ensure a fair test environment.

A. Dijkstra's Algorithm

- **How it works:** Dijkstra explores uniformly in all directions, evaluating the cumulative edge weights. It uses the custom Min-Heap to always explore the "cheapest" path available.
- **Implementation:** It maintains a dist array initialized to infinity. As it discovers nodes, it updates their distances. Once the dest node is extracted from the Min-Heap, the algorithm terminates, guaranteeing the absolute shortest path.

B. A* Search

- **How it works:** A* is an upgrade to Dijkstra. Instead of radiating in a perfect circle, it is "pulled" toward the destination.
- **Implementation:** It calculates an fScore which equals the actual distance traveled plus a *Heuristic*. The heuristic used is the **Euclidean Distance** formula: $\sqrt{(x_2 - x_1)^2 +

(y_2 - y_1)^2}$. By guessing the remaining distance, A* heavily prunes the search space, skipping nodes that are moving away from the target.

C. Breadth-First Search [BFS]

- **How it works:** BFS ignores edge weights completely. It checks all nodes 1 hop away, then all nodes 2 hops away, and so on.
- **Implementation:** It utilizes a standard First-In-First-Out (std::queue). It guarantees the shortest path *by number of edges*, but not necessarily by weight.

D. Depth-First Search [DFS]

- **How it works:** DFS goes as deep as possible down a single path until it hits a dead end, then backtracks just enough to try a new path.
- **Implementation:** It utilizes a recursive helper function (dfsHelper) and a simple boolean visited array. It makes no attempt to find the shortest path; it only cares about finding *any* path.

6. Microsecond Benchmarking Suite

A major engineering challenge in this project was accurate timekeeping. Modern CPUs are so fast that on a 30-node graph, C++ algorithms finish in less than 1 millisecond. Standard operating system clocks only "tick" every 15.6 milliseconds, resulting in an output of 0.000 ms.

- **The Solution (Loop Averaging):** The Benchmarker class was re-engineered to force each algorithm to run **1,000 times back-to-back** before stopping the std::chrono::high_resolution_clock.
- **The Math:** By taking the total block execution time and dividing it by 1,000, the system successfully bypasses the hardware clock limit, yielding true, hyper-accurate execution times displayed in microseconds (μs).

7. The Frontend Visualizer & Canvas Rendering

The interface was built using HTML5 Canvas and CSS Grid to provide a seamless analytical environment.

- **1:1 Pixel Coordinate Mapping:** HTML5 Canvases suffer from a "stretching" bug if the CSS size doesn't match the internal resolution. The JavaScript resizeCanvas() function dynamically reads the browser's bounding box and adjusts the internal pixel density to perfectly match the user's screen.
- **The Animation Engine:** The C++ backend executes instantly. To visualize this, the JS takes the final array of visitedNodes and mathematically reconstructs a step-by-step traversal. The user can manipulate an interactive slider to control the animation speed (from 20ms/step to 500ms/step)

8. Results & Interpretation

The true power of this project lies in observing the visual dashboard and the benchmark charts side-by-side. Here is exactly how to interpret the results generated by the software:

Visual Interpretation (The Canvas)

When the "Animate" button is pressed, users will observe two distinct behaviors:

1. **The Spreading Puddle:** Dijkstra, A*, and BFS will light up dozens of nodes in a cluster (the search phase), but eventually, only a few thick, straight lines are drawn. This visually proves that they evaluate many options but ultimately isolate a highly efficient, direct route.
2. **The Wandering Snake:** DFS will draw thick lines between almost every single node it lights up. Because it blindly wanders, its final "winning path" is a massive, tangled web. This visually proves that DFS is incredibly inefficient for spatial navigation.

Quantitative Interpretation (The Benchmark Panel)

When looking at the generated data for a small, 30-node graph, a fascinating computational anomaly appears:

- **Fastest Execution: BFS (approx. ~3.5 μs)** and **DFS (approx. ~4.4 μs)**.
    - *Interpretation:* Because they ignore edge weights, they require zero complex mathematics or sorting logic. Pushing integers into standard arrays and queues is incredibly fast for a CPU.
- **Mid-Tier Execution: Dijkstra (approx. ~5.5 μs)**.
    - *Interpretation:* Even though it finds the optimal path, it is slightly slower because the CPU must constantly re-sort the custom Min-Heap to keep the weights perfectly organized.
- **The Heuristic Paradox: A Search (approx. ~7.5 μs)*.**
    - *Interpretation:* Visually, A* visits the fewest nodes (e.g., 5 nodes compared to Dijkstra's 24). However, it takes the *longest* time. Why? Because A* calculates a square root (std::sqrt) for every neighbor. On a tiny graph, the CPU time required to calculate those square roots is actually greater than the time saved by skipping nodes.
    - *The Scaling Rule:* If the user generates a massive 10,000-node graph via the CLI, this metric completely flips. A* becomes magnitudes faster than Dijkstra, because skipping thousands of nodes vastly outweighs the cost of the square root mathematics.

9. Conclusion

The Full-Stack Graph Algorithm Engine successfully fulfills its objective as an advanced educational and analytical tool. By integrating custom low-level data structures with high-precision benchmarking and dynamic web rendering, the project clearly demystifies algorithmic behavior. It proves that there is no singular "best" algorithm; rather, developers must choose between memory efficiency (DFS), structural speed (BFS), absolute accuracy (Dijkstra), or heuristic optimization (A*) based on the exact scale and nature of the dataset.